The Rise and Fall of Binary Exploitation

Stephen Sims
DEF CON 32
@Steph3nSims

About Me

- Vulnerability Researcher
  - Exploit Sales:
    - 29 ~~Over 30~~ browser exploits (UAF and Type Confusion)
    - 3 Windows Kernel exploits – 1 RCE, 2 LPE
    - Many other vulnerabilities in commercial hardware and applications (Mobile, VPN Concentrators, AV/EDR, Gaming Consoles, etc…)
- Musician – deadcode – https://www.d3adc0de.com/
- Offensive Operations Curriculum Lead at SANS Institute
- Coauthor of Gray Hat Hacking (4th, 5th, & 6th editions) 7th?
- Author, Instructor, and Speaker
- Off By One Security – Weekly Stream

OFFBYONE SANS DEADCORE

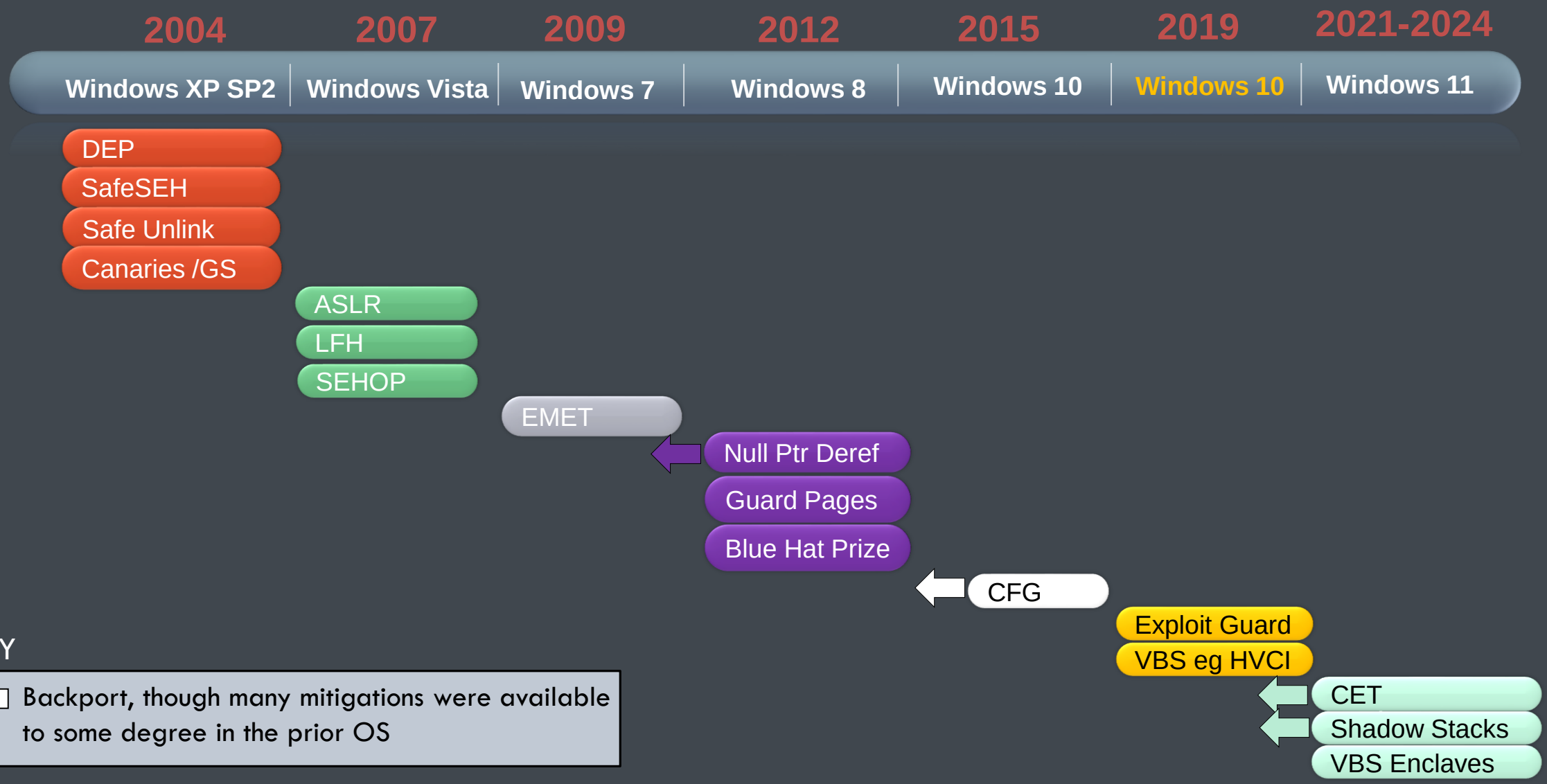
Agenda…

1. Brief history of mitigations…
2. Microsoft CVE's by bug class percentages
3. Payouts
4. The role of machine learning (ML) in vulnerability research (VR)
5. Analyzing a mitigation in the kernel
6. Take-aways…

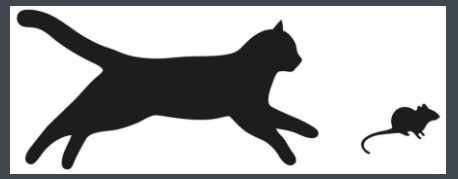
The Golden Years of Binary Exploitation

- Typically defined as the time before effective exploit mitigations
- Windows
  - 2004 – XP Service Pack 2 - Huge game-changer…
    - Data Execution Prevention (DEP)
    - Safe Structured Exception Handling (SafeSEH)
    - Security Cookies (Canaries)
    - Safe Unlink
  - 2007 – Address Space Layout Randomization (ASLR) Vista
  - *WehnTrust from Skape and wag predated EMET
- Linux
  - 2001 – ASLR *PaX
  - 2000 – PAGEXEC / MPROTECT
  - 1997 – StackGuard Canaries *GCC Patches
  - 1997 – Openwall Kernel patch (Solar Designer)
  - Much is owed to Openwall, grsecurity, and the PaX team (literally 1 or 2 amazing people)…
- macOS/Mac OS X
  - 2007 – DEP OS X 10.5 Leopard
  - 2011 – ASLR OS X Leopard
  - Late to the game…

High-Level Timeline – Some Notable Mitigations



A Cat and Mouse Game



2004 | 2007 | 2009 | 2012 | 2015 | 2019 | 2021-2024

Windows XP SP2 | Windows Vista | Windows 7 | Windows 8 | Windows 10 | Windows 10 | Windows 11

DEP
SafeSEH
Safe Unlink
Canaries /GS

Code reuse and ROP
Find unprotected module or memory segment
The Houses – Malloc Maleficarum, T-Cache bin attacks
Trigger SEH, repair, information leak

ASLR
LFH
SEHOP

Memory leak, non-rebased module, heap spray
Heap grooming, back-end allocator attacks
Solid mitigation… Limited bypass options, bug class mostly killed

EMET

<5% adoption rate, bypasses

Null Ptr Deref

Bug class killed

Guard Pages

Avoid them, heap grooming

Blue Hat Prize

Re-Pivot stack pointer, JOP

JIT, overwrite return pointer, bitmap ranges
Many are off by default

CFG

Exploit Guard
VBS eg HVCI

Hyper-V Flaws

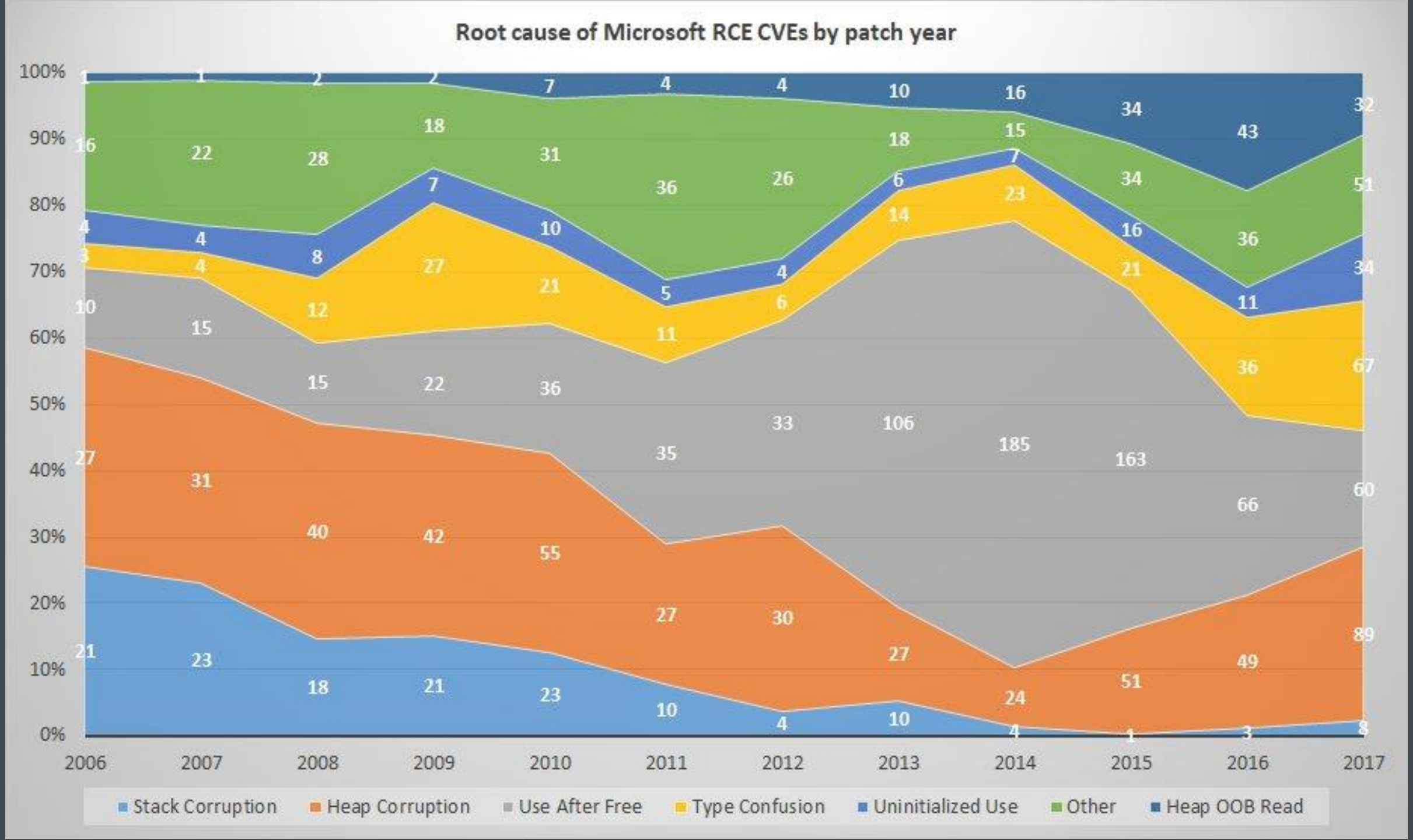
Execute only signed code or r/w

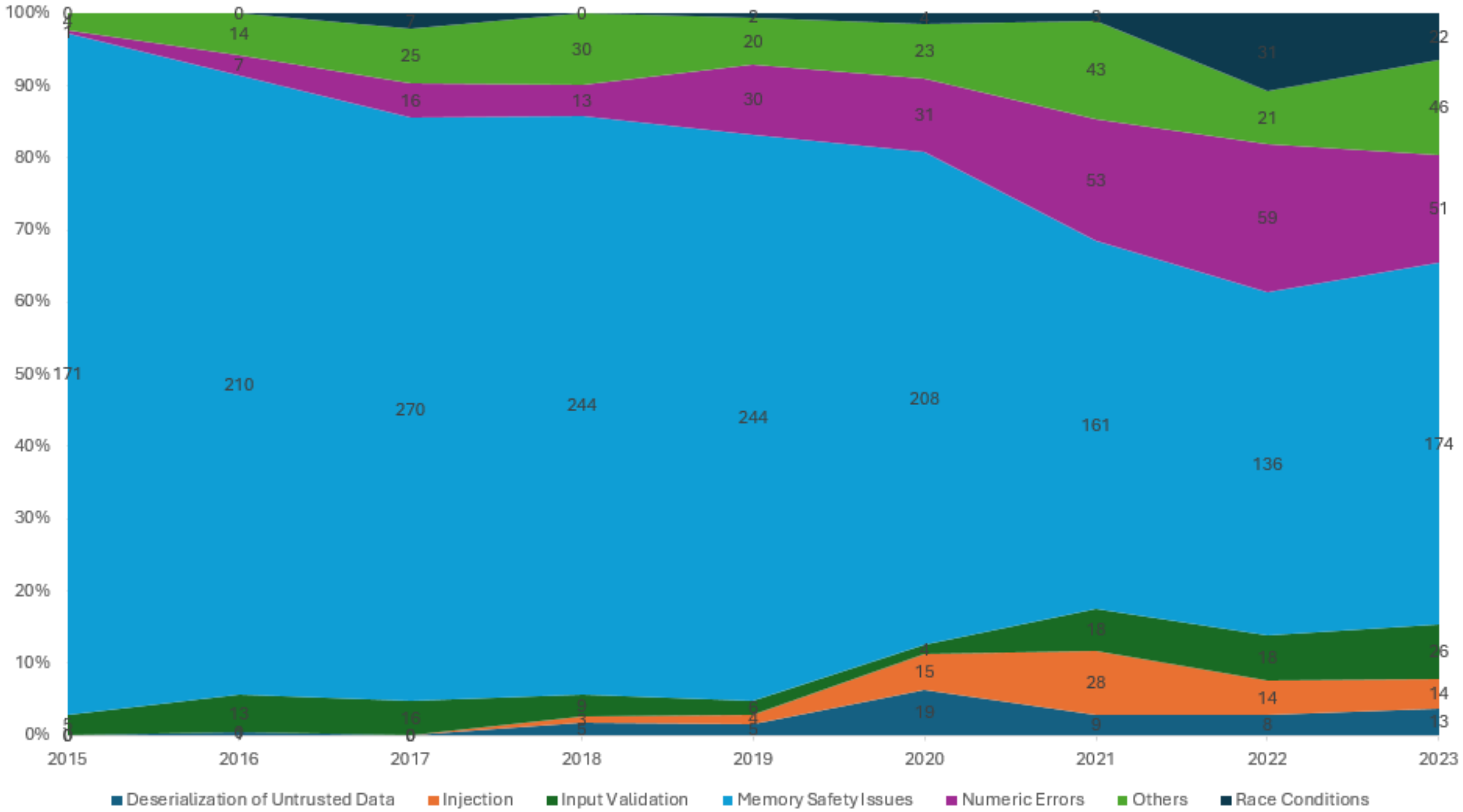
CET
Shadow Stacks
VBS Enclaves

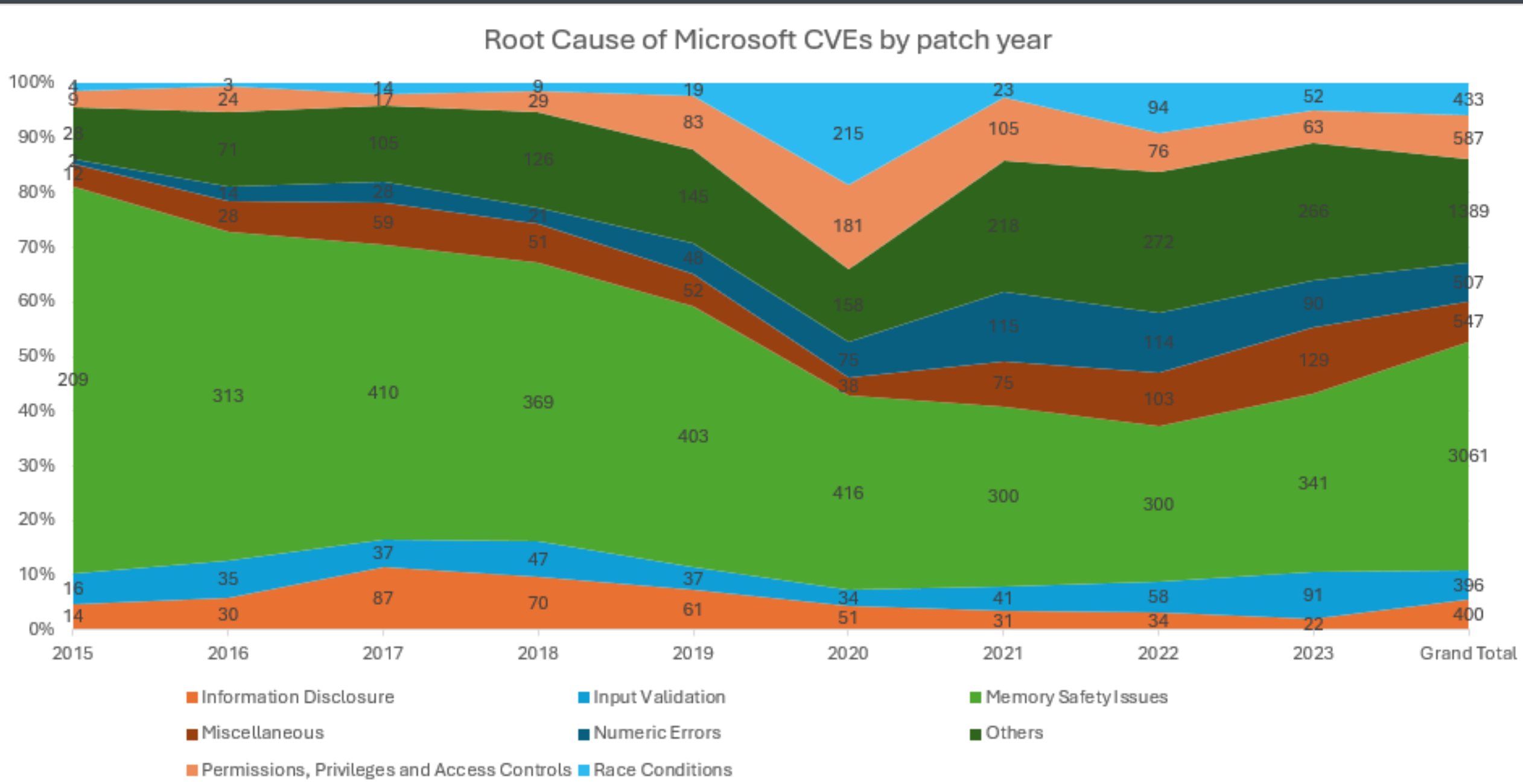
Side-Channel attacks
Desynchronization

KEY

⇦ Backport, though many mitigations were available to some degree in the prior OS


Root cause of Microsoft RCE CVEs by patch year
100%
90%
80%
70%
60%
50%
40%
30%
20%
10%
0%
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
Stack Corruption
Heap Corruption
Use After Free
Type Confusion
Uninitialized Use
Other
Heap OOB Read


Root Cause of Microsoft RCE CVEs by patch year
100%
90%
80%
70%
60%
50%
40%
30%
20%
10%
0%
2015
2016
2017
2018
2019
2020
2021
2022
2023
Deserialization of Untrusted Data
Injection
Input Validation
Memory Safety Issues
Numeric Errors
Others
Race Conditions


Root Cause of Microsoft CVEs by patch year
100%
90%
80%
70%
60%
50%
40%
30%
20%
10%
0%
2015
2016
2017
2018
2019
2020
2021
2022
2023
Grand Total
Information Disclosure
Input Validation
Memory Safety Issues
Miscellaneous
Numeric Errors
Others
Permissions, Privileges and Access Controls
Race Conditions

How Mitigations Have Affected Payouts

- First and foremost, there are several categories of buyers
  - Vendor disclosure program e.g. Microsoft, Apple, Google

How Mitigations Have Affected Payouts

- First and foremost, there are several categories of buyers
  - Vendor disclosure program e.g. Microsoft, Apple, Google
  - 3rd-Party disclosure programs e.g. ZDI, Bugcrowd, HackerOne, iDefense (RIP)

How Mitigations Have Affected Payouts

- First and foremost, there are several categories of buyers
  - Vendor disclosure program e.g. Microsoft, Apple, Google
  - 3rd-Party disclosure programs e.g. ZDI, Bugcrowd, HackerOne, iDefense (RIP)
  - Intelligence agency proxy companies e.g. The weird looking buildings around DC

How Mitigations Have Affected Payouts

- First and foremost, there are several categories of buyers
  - Vendor disclosure program e.g. Microsoft, Apple, Google
  - 3rd-Party disclosure programs e.g. ZDI, Bugcrowd, HackerOne, iDefense (RIP)
  - Intelligence agency proxy companies e.g. The weird looking buildings around DC
  - Exploit acquisition companies e.g. Crowdfense, VulnPoint

How Mitigations Have Affected Payouts

- First and foremost, there are several categories of buyers
  - Vendor disclosure program e.g. Microsoft, Apple, Google
  - 3rd-Party disclosure programs e.g. ZDI, Bugcrowd, HackerOne, iDefense (RIP)
  - Intelligence agency proxy companies e.g. The weird looking buildings around DC
  - Exploit acquisition companies e.g. Crowdfense, VulnPoint
  - Independent, individual proxy buyers e.g. “Randos” on Telegram, TOR

How Mitigations Have Affected Payouts

- First and foremost, there are several categories of buyers
  - Vendor disclosure program e.g. Microsoft, Apple, Google
  - 3rd-Party disclosure programs e.g. ZDI, Bugcrowd, HackerOne, iDefense (RIP)
  - Intelligence agency proxy companies e.g. The weird looking buildings around DC
  - Exploit acquisition companies e.g. Crowdfense, VulnPoint
  - Independent, individual proxy buyers e.g. “Randos” on Telegram, TOR

- Example 1 – A UAF or Type Confusion Edge RCE Exploit
  - 2014 – $10K to $30K USD (MemGC mitigated most as of late 2014) – Source: iDefense
  - 2024 – $400K – Source: Crowdfense Bounty Program

How Mitigations Have Affected Payouts

- First and foremost, there are several categories of buyers
  - Vendor disclosure program e.g. Microsoft, Apple, Google
  - 3rd-Party disclosure programs e.g. ZDI, Bugcrowd, HackerOne, iDefense (RIP)
  - Intelligence agency proxy companies e.g. The weird looking buildings around DC
  - Exploit acquisition companies e.g. Crowdfense, VulnPoint
  - Independent, individual proxy buyers e.g. “Randos” on Telegram, TOR

- Example 1 – A UAF or Type Confusion Edge RCE Exploit
  - 2014 – $10K to $30K USD (MemGC mitigated most as of late 2014) – Source: iDefense
  - 2024 – $400K – Source: Crowdfense Bounty Program
- Example 2 – iOS Zero-Click, RCE/LPE/SBX/FCP
  - 2014 – $2M USD (BlastDoor introduced in 2021 with iOS 14.4) Source: Zerodium
  - 2024 – $9M+ USD – Source: Crowdfense, VulnPoint

How Mitigations Have Affected Payouts

- First and foremost, there are several categories of buyers
  - Vendor disclosure program e.g. Microsoft, Apple, Google
  - 3rd-Party disclosure programs e.g. ZDI, Bugcrowd, HackerOne, iDefense (RIP)
  - Intelli… …ldings around DC
  - Explo…
  - Indep… …m, TOR

  - Exam…
    - 2014 – $10K to $30K USD (MemGC mitigated most as of late 2014) – Source: iDefense
    - 2024 – $400K – Source: Crowdfense Bounty Program
  - Example 2 – iOS Zero-Click, RCE/LPE/SBX/FCP
    - 2014 – $2M USD (BlastDoor introduced in 2021 with iOS 14.4) Source: Zerodium
    - 2024 – $9M+ USD – Source: Crowdfense, VulnPoint

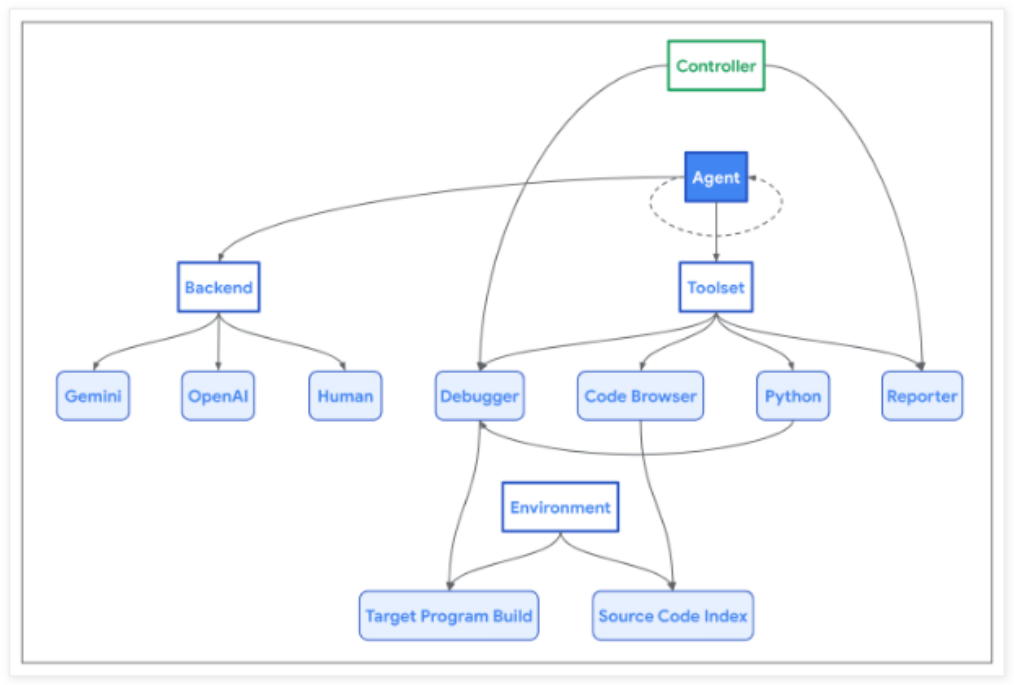
Machine Learning for Vulnerability Research

- Google Project Zero's efforts to develop AI-assisted advanced memory corruption vulnerability research

> *"While we have shown that principled agent design can greatly improve the performance of general-purpose LLMs on challenges in the security domain, it's the opinion of the Project Zero team that substantial progress is still needed before these tools can have a meaningful impact on the daily work of security researchers."*[1]

- Other research groups using agents each focused on a link in the vulnerability research chain

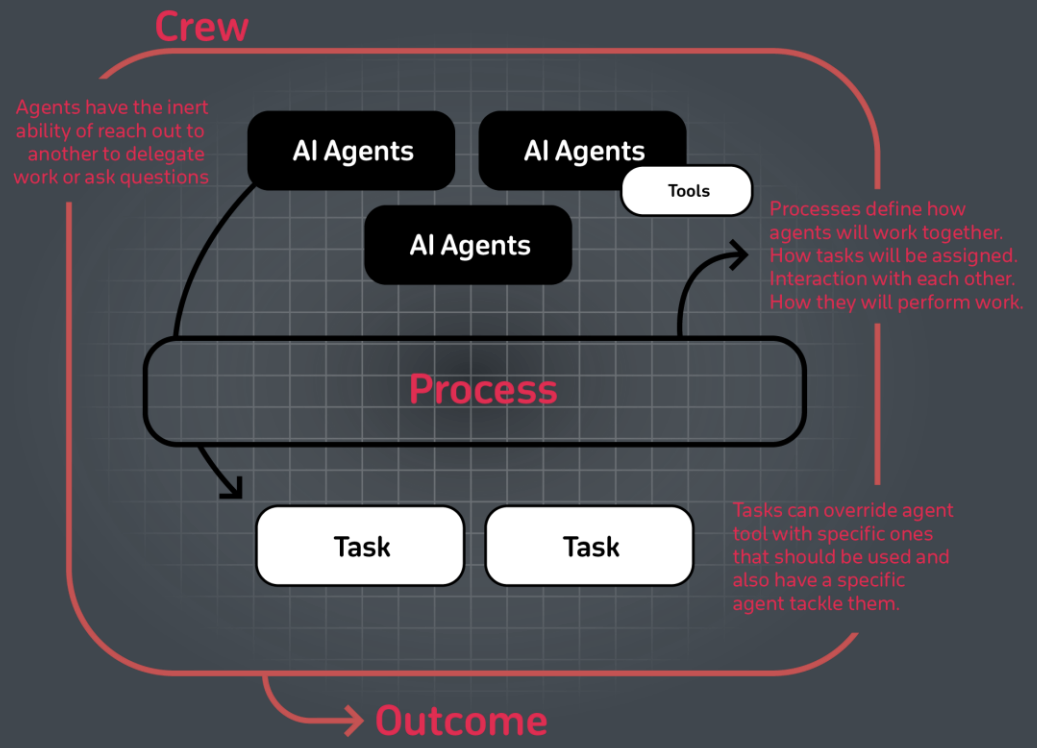
Project Naptime: Evaluating Offensive Security Capabilities of Large Language Models



[1] **SOURCE:** https://googleprojectzero.blogspot.com/2024/06/project-naptime.html

Leveraging AI Agents – Typical Non-Vulnerability Research Example (1)

The currently demonstrated architecture relies on a series of **stuffed prompts** automated through a SOAR platform. Context is provided to the LLM by adding previous command inputs and results in the prompts. There are better ways:



SOURCE: https://github.com/joaomdmoura/crewAI

Example "Threat Intelligence Analyst" agent definition:

```
agent = Agent(
  role='Threat Intelligence Analyst',
  goal='Extract MITRE ATT&CK TTPs from reports',
  backstory="""You're a threat intelligence analyst at the best
  security services provider in the world. You're responsible for
  analyzing threat intelligence reports and extracting MITRE ATT&CK
  TTPs that can be used to build adversary emulation campaigns.""",
  tools=[my_tool1, my_tool2],
  llm=my_llm,
  function_calling_llm=my_llm,
  max_iter=10,
  max_rpm=10,
  verbose=True,
  allow_delegation=True,
  step_callback=my_intermediate_step_callback
)
```

Leveraging AI Agents – Typical Non-Vulnerability Research Example (2)

"In the CrewAI framework, an agent is considered a member of a team, each with **specific skills** and **responsibilities**. Agents can perform tasks according to their roles, such as researchers, writers, or customer support, each contributing to the overall goal of the team."

SOURCE: https://github.com/joaomdmoura/crewAI

Threat Intelligence Analyst

Input
Threat intel reports (PDFs)

Activities
Analyze threat intel report

Output
List of MITRE ATT&CK TTPs

Red Team Operator

Input
MITRE ATT&CK TTPs

Activities
Design abilities to test TTPs

Output
Caldera abilities

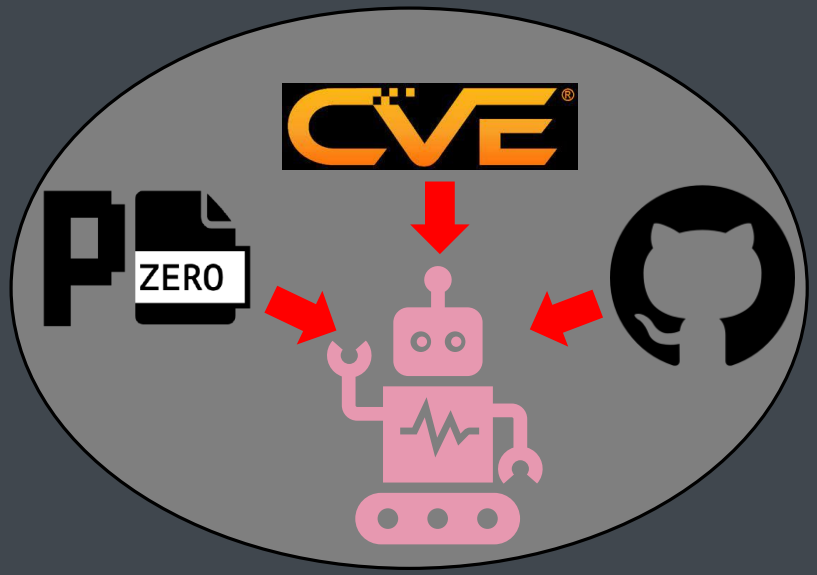
Detection Engineer

Input
MITRE ATT&CK TTPs

Activities
Design TTP detections

Output
Detection rules

Vulnerability Research: Leveraging AI Agents for Exploit Dev

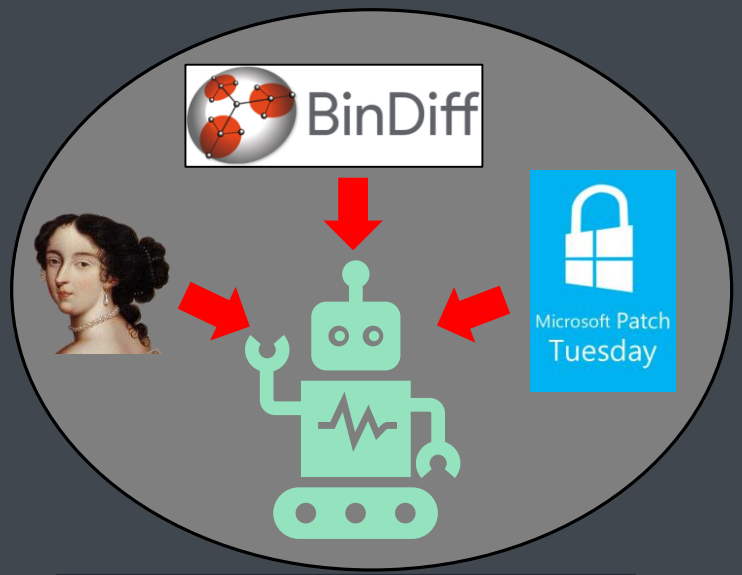


VR Data Collector Agent

Input
Vulnerability Research

Activities
Learn all prior 0-day and n-day causes and exploits

Output
List of all vulnerabilities

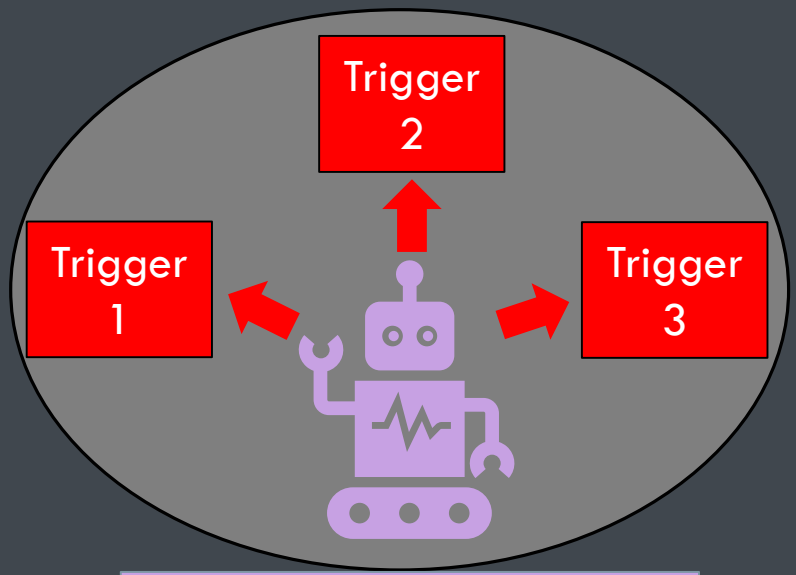


Patch Diff Agent

Input
Microsoft Patches / vulnerability list

Activities
Diff patches and compare against prior agent data

Output
Vulnerabilities & patch delta



Corpus Generation Agent

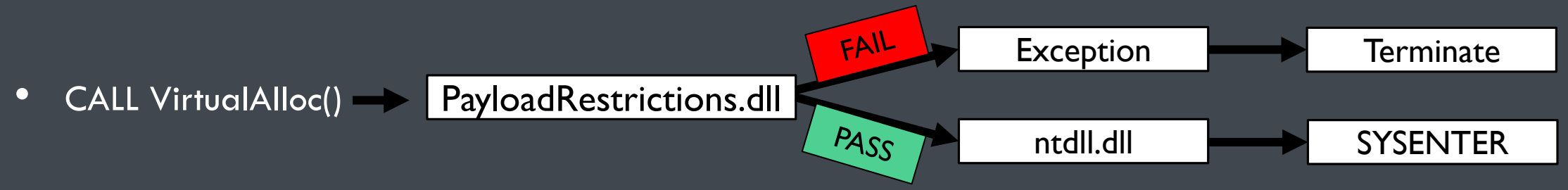
Input
Patch delta and vulnerability guess

Activities
Generate corpus for fuzzer

Output
Trigger files

Understanding How Mitigations Work – Exploit Guard

- The module PayloadRestrictions.dll is loaded into all processes designated for protection by Exploit Guard
- Many of the controls simply "hook" application flow at specific points
  - An example of hooking is when a table of pointers to various functions is overwritten with pointers to different code, or hot-patched…
    - This is commonly used by malware, endpoint protection suites, and anti-exploitation products
    - Typically, the originally intended function is reached after going through a series of checks



CreateProcess.cpp – PoC Program to Test Mitigation

- Simple program to create whatever process entered as an argument
- Additional code to call VirtualAlloc() for other mitigations such as Export Address Table Filtering (EAF)

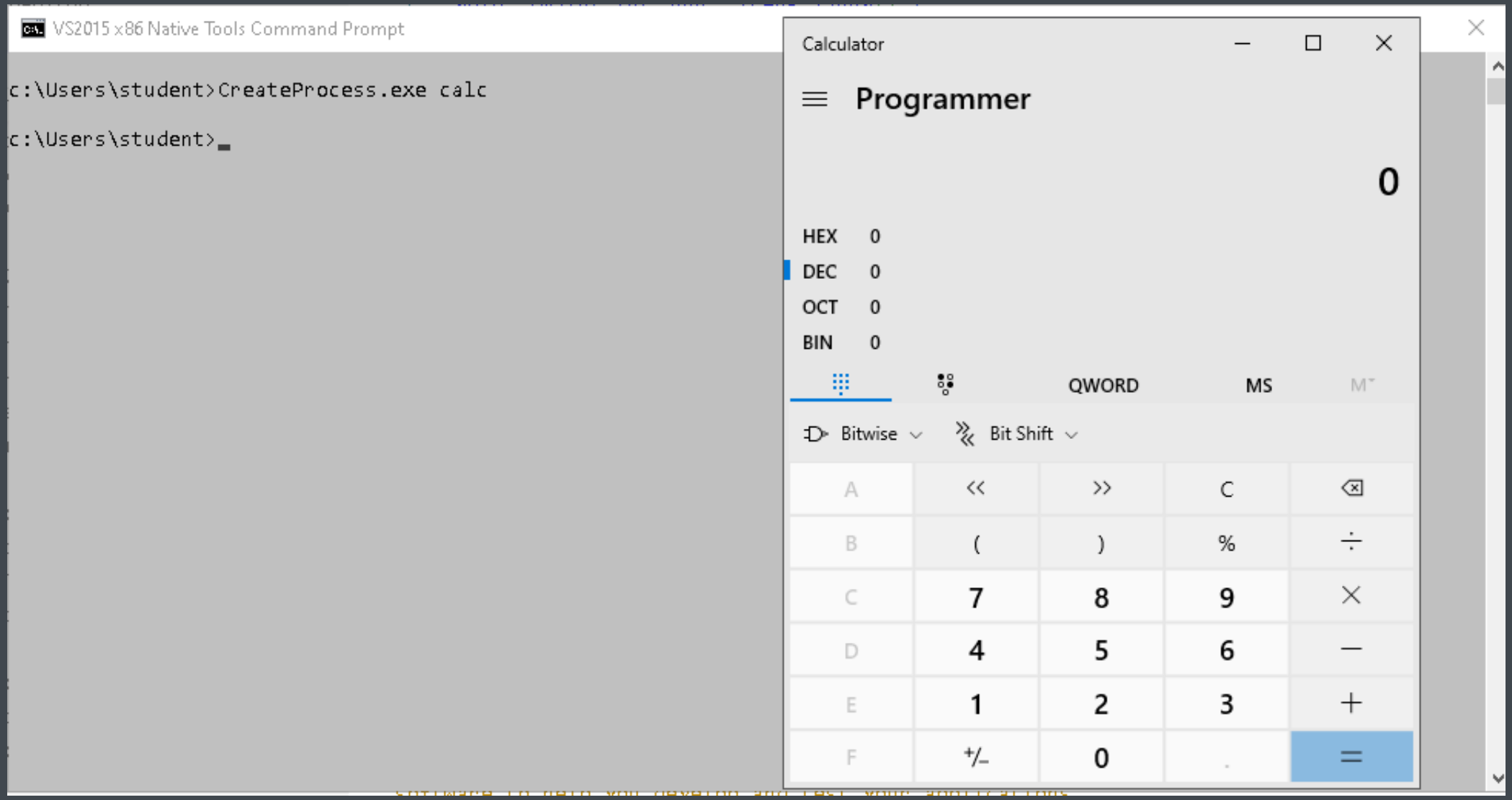
```
c:\Users\student>cl CreateProcess.cpp
Microsoft (R) C/C++ Optimizing Compiler Version 19.00.24210 for x86
Copyright (C) Microsoft Corporation.  All rights reserved.

/out:CreateProcess.exe
CreateProcess.obj
```

```
7   void _tmain( int argc, TCHAR *argv[] )
8   {
9       STARTUPINFO si;
10      PROCESS_INFORMATION pi;
11
12      ZeroMemory( &si, sizeof(si) );
13      si.cb = sizeof(si);
14      ZeroMemory( &pi, sizeof(pi) );
15
16      if( argc == 2 ) {
17      // Start the child process.
18      if( !CreateProcess( NULL,   // No module name (use command line)
19          argv[1],        // Command line
20          NULL,           // Process handle not inheritable
21          NULL,           // Thread handle not inheritable
22          FALSE,          // Set handle inheritance to FALSE
23          0,              // No creation flags
24          NULL,           // Use parent's environment block
25          NULL,           // Use parent's starting directory
26          &si,            // Pointer to STARTUPINFO structure
27          &pi )           // Pointer to PROCESS_INFORMATION structure
28      )
29      {
30          printf( "CreateProcess failed (%d).\n", GetLastError() );
31          return;
32      }
33      }
34      else{
35          Sleep(30000);
36          void *pPage = VirtualAllocEx(GetCurrentProcess(), NULL, 256, MEM_RESERVE | MEM_COMMIT, PAGE_EXECUTE_READWRITE);
37          DWORD floldprotect = 0;
38          VirtualProtect(pPage, 256, PAGE_EXECUTE_READ, &floldprotect);
39  }
40      // Wait until child process exits.
41      WaitForSingleObject( pi.hProcess, INFINITE );
42
43      // Close process and thread handles.
44      CloseHandle( pi.hProcess );
45      CloseHandle( pi.hThread );
46  }
```

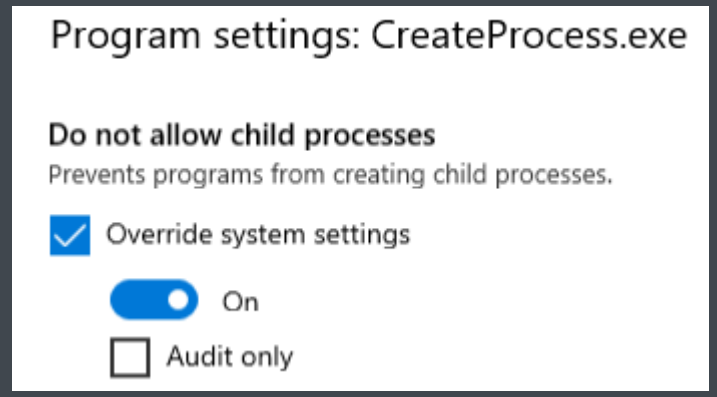
https://learn.microsoft.com/en-us/windows/win32/procthread/creating-processes

Running the Program with the Mitigation Off

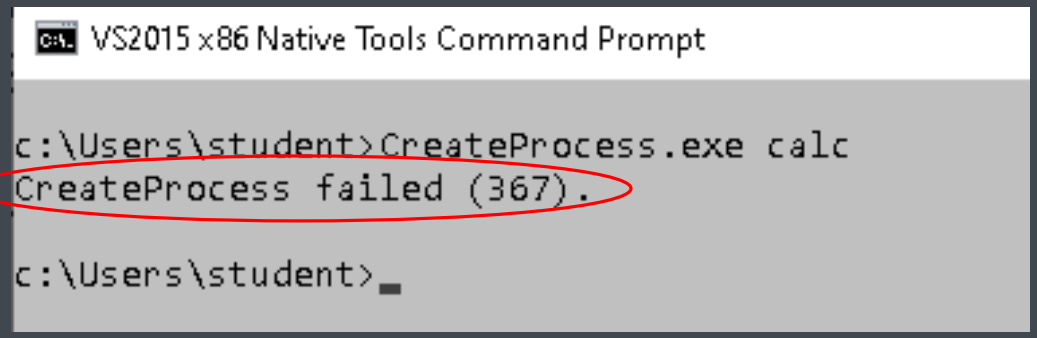


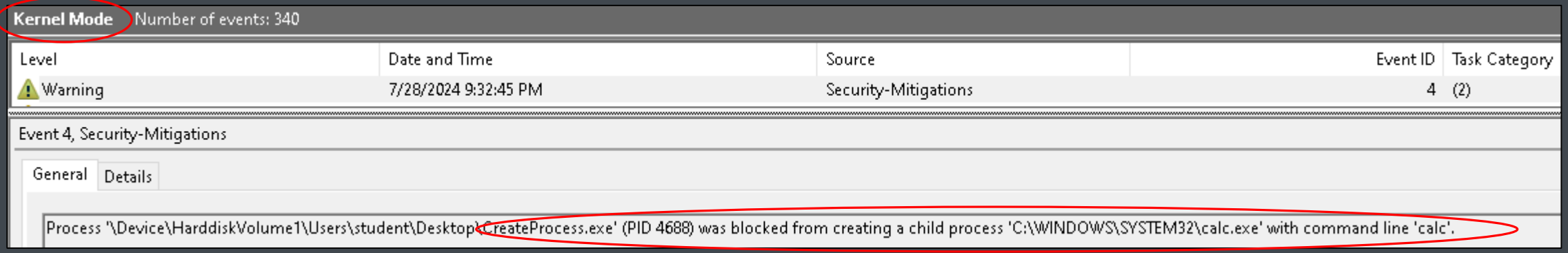
Running the Program with the Mitigation On

- The program was added to Exploit Guard with the “Do not allow child processes” mitigation added



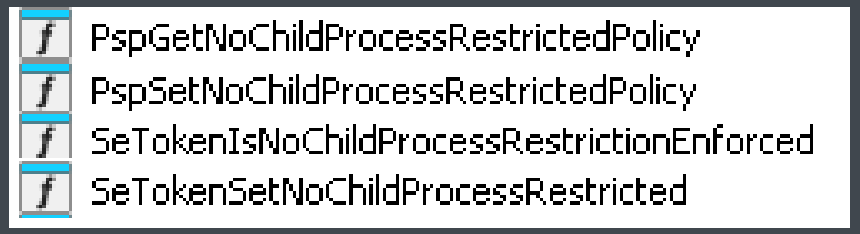
- Calc now fails to spawn
- How is the mitigation being enforced?





Looking at the NTOS Kernel

- When setting a breakpoint on the call to the CreateProcess function with the mitigation on, PayloadRestrictions.dll (Exploit Guard) does not execute any code and the syscall is made into the Kernel
- The following functions in IDA stick out as being related to the mitigation:



- Let's keep an eye out for them soon and set a breakpoint on the Kernel equivalent function for CreateProcess():

```
kd> bp nt!NtCreateUserProcess
```

- The breakpoint is immediately hit when running: `c:\Users\student>CreateProcess.exe calc`

```
Breakpoint 0 hit
nt!NtCreateUserProcess:
fffff800`76c2e480 4055            push    rbp
```

Finding the Current Process (1)

- Let's validate that we're in the right process by looking at the current thread, as CreateProcess() is called often

```
3: kd> uf nt!PsGetCurrentThread
nt!KeGetCurrentThread:
fffff800`76631e40 65488b042588010000 mov   rax,qword ptr gs:[188h]
fffff800`76631e49 c3                 ret
```

The GS segment register points to Kernel Processor Control Region (KPCR)

- GS:[188h] is being dereferenced… Let's look at the structure on the Vergilius Project of KPCR
- It's 8-bytes into KPRCB
- At that offset is a pointer to the KTHREAD structure of the current thread:

```
struct _KTHREAD* CurrentThread; //0x8
```

```
struct _KPCR
{
    union
    {
        struct _NT_TIB NtTib;                          //0x0
        struct
        {
            union _KGDTENTRY64* GdtBase;               //0x0
            struct _KTSS64* TssBase;                   //0x8
            ULONGLONG UserRsp;                         //0x10
            struct _KPCR* Self;                        //0x18
            struct _KPRCB* CurrentPrcb;                //0x20
            struct _KSPIN_LOCK_QUEUE* LockArray;       //0x28
            VOID* Used_Self;                           //0x30
        };
    };
    union _KIDTENTRY64* IdtBase;                       //0x38
    ULONGLONG Unused[2];                               //0x40
    UCHAR Irql;                                        //0x50
    UCHAR SecondLevelCacheAssociativity;               //0x51
    UCHAR ObsoleteNumber;                              //0x52
    UCHAR Fill0;                                       //0x53
    ULONG Unused0[3];                                  //0x54
    USHORT MajorVersion;                               //0x60
    USHORT MinorVersion;                               //0x62
    ULONG StallScaleFactor;                            //0x64
    VOID* Unused1[3];                                  //0x68
    ULONG KernelReserved[15];                          //0x80
    ULONG SecondLevelCacheSize;                        //0xbc
    ULONG HalReserved[16];                             //0xc0
    ULONG Unused2;                                     //0x100
    VOID* KdVersionBlock;                              //0x108
    VOID* Unused3;                                     //0x110
    ULONG PcrAlign1[24];                               //0x118
    struct _KPRCB Prcb;                                //0x180
};
```

Finding the Current Process (2)

- Let's run a command in WinDbg to check our current process:

```
3: kd> !process @@(@$prcb->CurrentThread->ApcState.Process) 0
PROCESS ffffdf094214e080
    SessionId: 1  Cid: 071c    Peb: 4b39b97000  ParentCid: 140c
    DirBase: 103e7a002  ObjectTable: ffffb08e6fab9640  HandleCount:  72.
    Image: cmd.exe

3: kd> g
Breakpoint 0 hit
nt!NtCreateUserProcess:
fffff800`76c2e480 4055            push    rbp
1: kd> !process @@(@$prcb->CurrentThread->ApcState.Process) 0
PROCESS ffffdf093fb464c0
    SessionId: 1  Cid: 05a0    Peb: f1b5bcf000  ParentCid: 071c
    DirBase: a79b3002  ObjectTable: ffffb08e6e99edc0  HandleCount:  36.
    Image: CreateProcess.exe
```

The first breakpoint hit was cmd.exe starting the CreateProcess.exe program

- Now that we're in the right process, let's continue…

Setting Breakpoints on the Mitigation Functions

- Let's set breakpoints on the two functions associated with setting and checking the Do Not Allow Child Processes mitigation:

```
kd> bl
  0 e Disable Clear  fffff800`76c2e480     0001 (0001) nt!NtCreateUserProcess
  1 e Disable Clear  fffff800`7669a424     0001 (0001) nt!SeTokenGetNoChildProcessRestricted
  2 e Disable Clear  fffff800`76732490     0001 (0001) nt!SeTokenSetNoChildProcessRestricted
```

- When restarting the process, the "SeTokenSetNoChildProcessRestricted" function is hit under the context of cmd.exe
- This sounds like the parent process calling CreateProcess() is responsible for turning the mitigation on in the child process
- Let's verify

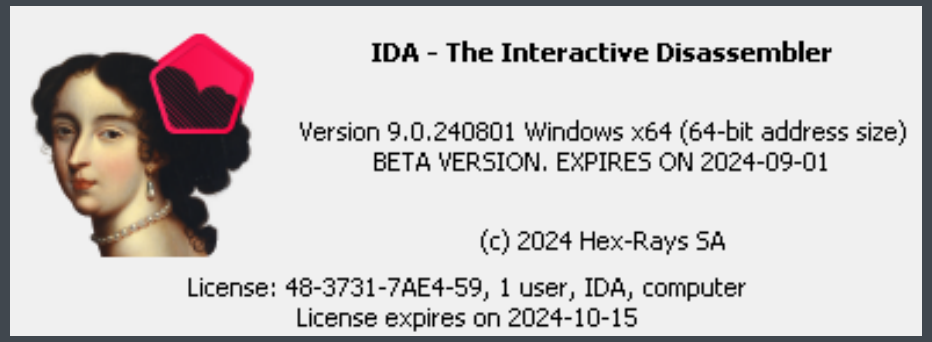
Tell Hex-Rays that it's a Token Struct - Before

```
void SeTokenSetNoChildProcessRestricted(
  [in] PACCESS_TOKEN Token,
  [in] BOOLEAN       UnlessSecure,
  [in] BOOLEAN       AuditOnly
);
```

```
__int64 __fastcall SeTokenSetNoChildProcessRestricted(__int64 a1, char a2, char a3)
{
  struct _KTHREAD *CurrentThread; // rax
  int v7; // edx
  int v8; // eax
  int v9; // edx
  unsigned int v10; // eax
  unsigned int v11; // edx
  signed __int32 v13[10]; // [rsp+0h] [rbp-28h] BYREF

  CurrentThread = KeGetCurrentThread();
  --CurrentThread->KernelApcDisable;
  ExAcquireResourceExclusiveLite(*(PERESOURCE *)(a1 + 48), 1u);
  _InterlockedOr(v13, 0);
  v7 = *(_DWORD *)(a1 + 200);
  if ( a3 )
  {
    if ( (v7 & 0x80000) != 0 )
      goto LABEL_6;
    v11 = v7 | 0x200000;
  }
  else
  {
    v8 = v7 | 0x80000;
    v9 = v7 | 0x180000;
    v10 = v8 & 0xFFEFFFFF;
    if ( !a2 )
      v9 = v10;
    v11 = v9 & 0xFFDFFFFF;
  }
  *(_DWORD *)(a1 + 200) = v11;
LABEL_6:
  *(_QWORD *)(a1 + 56) = ExpLuidIncrement + _InterlockedExchangeAdd64(&ExpLuid, ExpLuidIncrement);
  _InterlockedOr(v13, 0);
  ExReleaseResourceLite(*(PERESOURCE *)(a1 + 48));
  return KeLeaveCriticalRegionThread(KeGetCurrentThread());
}
```

IDA 9 allows for headless processing!



Tell Hex-Rays that it's a Token Struct - After

```
__int64 __fastcall SeTokenSetNoChildProcessRestricted(_TOKEN *Token, char UnlessSecure, char AuditOnly)
{
  struct _KTHREAD *CurrentThread; // rax
  unsigned int TokenFlags; // edx
  int v8; // eax
  int v9; // edx
  unsigned int v10; // eax
  unsigned int v11; // edx
  signed __int32 v13[10]; // [rsp+0h] [rbp-28h] BYREF

  CurrentThread = KeGetCurrentThread();
  --CurrentThread->KernelApcDisable;
  ExAcquireResourceExclusiveLite(Token->TokenLock, 1u);
  _InterlockedOr(v13, 0);
  TokenFlags = Token->TokenFlags;
  if ( AuditOnly )
  {
    if ( (TokenFlags & 0x80000) != 0 )
      goto LABEL_6;
    v11 = TokenFlags | 0x200000;
  }
  else
  {
    v8 = TokenFlags | 0x80000;
    v9 = TokenFlags | 0x180000;
    v10 = v8 & 0xFFEFFFFF;
    if ( !UnlessSecure )
      v9 = v10;
    v11 = v9 & 0xFFDFFFFF;
  }
  Token->TokenFlags = v11;
LABEL_6:
  Token->ModifiedId = (_LUID)(ExpLuidIncrement + _InterlockedExchangeAdd64(&ExpLuid, ExpLuidIncrement));
  _InterlockedOr(v13, 0);
  ExReleaseResourceLite(Token->TokenLock);
  return KeLeaveCriticalRegionThread(KeGetCurrentThread());
}
```

```
233             [SDKName("SPECIAL_ENCRYPTED_OPEN")]
234             SpecialEncryptedOpen = 0x40000,
235             [SDKName("TOKEN_NO_CHILD_PROCESS")]
236             NoChildProcess = 0x80000,
237             [SDKName("TOKEN_NO_CHILD_PROCESS_UNLESS_SECURE")]
238             NoChildProcessUnlessSecure = 0x100000,
239             [SDKName("TOKEN_AUDIT_NO_CHILD_PROCESS")]
240             AuditNoChildProcess = 0x200000,
241             [SDKName("TOKEN_PERMISSIVE_LEARNING_MODE")]
242             PermissiveLearningMode = 0x00400000,
```

https://github.com/googleprojectzero/sandbox-attacksurface-analysis-tools/blob/main/NtApiDotNet/NtTokenNative.cs

Let's Add Comments!

```
  CurrentThread = KeGetCurrentThread();
  --CurrentThread->KernelApcDisable;
  LOBYTE(v7) = 1;
  ExAcquireResourceExclusiveLite(Token->TokenLock, v7);// Acquire lock on Token
  _InterlockedOr(v13, 0);
  TokenFlags = Token->TokenFlags;               // Initialize TokenFlags variable with the TokenFlags value passed from the token object
  if ( AuditOnly )                              // If AuditOnly mode is True
  {
    if ( (TokenFlags & 0x80000) != 0 )          // If "No Child Process" is True, go to Label_6 (Bow Out)
      goto LABEL_6;
    v12 = TokenFlags | 0x200000;                // Set the AuditOnly bit in TokenFlags bitmask and put result in V12
  }
  else                                          // If AuditOnly is False
  {
    v9 = TokenFlags | 0x80000;                  // Set the NoChildProcess bit in TokenFlags bitmask and put result in v9
    v10 = TokenFlags | 0x180000;                // Set the NoChildProcessUnlessSecure and NoChildProcess bits in TokenFlags variable and put results in v10
    v11 = v9 & 0xFFEFFFFF;                      // Clear NoChildProcessUnlessSecure bit in v11
    if ( !UnlessSecure )                        // If NoChildProcessUnlessSecure is not set, clear bit in v10
      v10 = v11;                                // //
    v12 = v10 & 0xFFDFFFFF;                     // Clear AuditOnly bit in v12
  }
  Token->TokenFlags = v12;                      // Update the TokenFlags field in the Token object with changes
LABEL_6:
  Token->ModifiedId = (ExpLuidIncrement + _InterlockedExchangeAdd64(&ExpLuid, ExpLuidIncrement));// Increment the Token's ModifiedId member for version tracking
  _InterlockedOr(v13, 0);
  ExReleaseResourceLite(Token->TokenLock);      // Release lock on Token
  KeLeaveCriticalRegionThread(KeGetCurrentThread());
}
```

What We've Learned…

- The enforcement settings in relation to the "Do Not Allow Child Processes" mitigation are set by the parent process
  - e.g. In our case, cmd.exe was used to create the CreateProcess.exe program
  - When cmd.exe created the CreateProcess.exe process, the SeTokenSetNoChildProcessRestricted() function was called to turn on the bit in the TokenFlags bitmask to prevent CreateProcess.exe from being able to create a child process
  - When a process with the mitigation enforced attempts to spawn a child process, the SeTokenGetNoChildProcessRestricted() function is called to check the associated bit in the TokenFlags field, resulting in the failure to spawn the process, as well as the creation of a log entry
- How might we bypass this protection?
  - Impersonate the anonymous token? e.g. Call NtImpersonateAnonymousToken…
    - https://www.exploit-db.com/exploits/44888
  - Windows Management Instrumentation (WMI)? e.g. Use WMI or PS to execute under current process
  - Process Migration?
  - Reflexive DLL Loading?

Interestingly… TokenFlags is unrelated to the MitigationFlags Fields in EPROCESS

MitigationFlags in the EPROCESS

Since the introduction of Control Flow Guard for version 6.3, Windows accumulated rather many bit fields for whether this or that security mitigation applies. They were sprinkled through the **Flags**, **Flags2** and **Flags3** members of the **EPROCESS**. The 1709 release of Windows 10 collects them into a new set in union with a **ULONG** member named **MitigationFlags**, and added more—indeed, enough to overflow into a **MitigationFlags2**.

Mask	Definition	Versions
0x00000001	`ULONG ControlFlowGuardEnabled: 1;`	1709 and higher
0x00000002	`ULONG ControlFlowGuardExportSuppressionEnabled : 1;`	1709 and higher
0x00000004	`ULONG ControlFlowGuardStrict : 1;`	1709 and higher
0x00000008	`ULONG DisallowStrippedImages : 1;`	1709 and higher
0x00000010	`ULONG ForceRelocateImages : 1;`	1709 and higher
0x00000020	`ULONG HighEntropyASLREnabled : 1;`	1709 and higher
0x00000040	`ULONG StackRandomizationDisabled : 1;`	1709 and higher
0x00000080	`ULONG ExtensionPointDisable : 1;`	1709 and higher
0x00000100	`ULONG DisableDynamicCode : 1;`	1709 and higher

MitigationFlags2 in the EPROCESS

Since the introduction of Control Flow Guard for version 6.3, Windows accumulated rather many bit fields for whether this or that security mitigation applies. The 1709 release of Windows 10 collected many of these from the **Flags**, **Flags2** and **Flags3** members of the **EPROCESS** for a new set in union with a **ULONG** member named **MitigationFlags**. New bits for new mitigations are mostly in a second new set in union with a **ULONG** named **MitigationFlags2**.

Mask	Definition	Versions
0x00000001	`ULONG EnableExportAddressFilter : 1;`	1709 and higher
0x00002000	`ULONG SpeculativeStoreBypassDisable : 1;`	1809 and higher
0x00004000	`ULONG CetShadowStacks : 1;`	1809 only
	`ULONG CetUserShadowStacks : 1;`	1903 and higher
0x00008000	`ULONG AuditCetUserShadowStacks : 1;`	2004 and higher
0x00010000	`ULONG AuditCetUserShadowStacksLogged : 1;`	2004 and higher
0x00020000	`ULONG UserCetSetContextIpValidation : 1;`	2004 and higher
0x00040000	`ULONG AuditUserCetSetContextIpValidation : 1;`	2004 and higher
0x00080000	`ULONG AuditUserCetSetContextIpValidationLogged : 1;`	2004 and higher

Often, there are usermode and kernelmode hooks, combined with bitmask settings in kernel structures, registry settings, and others which make up a mitigation…

Take-Aways…

- The world of binary exploitation & vulnerability acquisition programs have much nuance…
- There are ethical and legal considerations, as well as many other colorful considerations…
- The golden years of binary exploitation may be in the past, but it seems we don't learn from the mistakes made, thus creating new attack surfaces as technology advances…
- Exploit mitigations can protect us, but…
  - It's important that we understand them, for both offensive and defensive purposes…
  - We can sometimes avoid them, or even disable or break them…
  - There are other concerns, such as overhead and technology limitations…
  - New mitigations drive the shift to alternative techniques…
- The price will continue to rise as the targets get more secure…
- Truly understanding how mitigations work at a low-level is how workarounds are realized

References and Thanks

- Jeremy Tinder (MSRC), David Weston (dwizzzle), & Matt Miller (Skape)
- Brad Spengler (grsecurity) - https://grsecurity.net/
- CrewAI - https://www.crewai.com/
- Geoff Chappell (RIP) ☹ - https://www.geoffchappell.com/studies/windows/km/ntoskrnl/inc/ntos/ps/eprocess/mitigationflags.htm
- Haroon Meer - @haroonmeer (and everyone from his 2010 BH talk)
- Google Project Zero - https://googleprojectzero.blogspot.com/
- Hex-Rays - https://hex-rays.com/
- Vergilius Project - https://www.vergiliusproject.com/
- Jonathan Reiter - @jon__reiter
- Pavel Yosifovich - https://github.com/zodiacon
- …and pwcrack (Bob Weiss) and the DEF CON CFP board for selecting my CFP submission!

QUESTIONS?

Stephen Sims

stephen@offbyonesecurity.com

@Steph3nSims

https://youtube.com/@OffByOneSecurity

https://discord.gg/offbyonesecurity

Feel free to reach out to me via one of these mediums…